Literature Review

PentraAI | AI-Driven Autonomous Web Penetration Testing

Prabanjan Velayutham · Pooja Sri Kuppuswamy Niranjana · Muhammad Saad Abdullah · Mohammed Iliyaz

Action Learning Program | Spring 2026

1. Problem Context

1.1 What Problem Are You Addressing?

Current web security relies on tools like Burp Suite, OWASP ZAP, and Nuclei that excel at detecting known vulnerability signatures such as SQL injection, cross-site scripting, and basic misconfigurations. However, these tools are fundamentally incapable of detecting vulnerability classes that require contextual reasoning: Insecure Direct Object References (IDOR/BOLA), Broken Authentication logic including JWT algorithm confusion and OAuth misconfigurations, and Race Conditions. These categories consistently appear at the top of the OWASP Top 10:2025 and are responsible for the most damaging real-world breaches, yet no mainstream tool automates their detection without expert human configuration at every step.

PentraAI targets three specific OWASP Top 10:2025 vulnerability categories that share one defining characteristic: their exploitation patterns are semantically indistinguishable from legitimate, well-formed application requests, making them invisible to pattern-matching scanners but exploitable by any adversary capable of contextual reasoning.

- **Insecure Direct Object References (IDOR/BOLA A01:2025):** Access control violations where one authenticated user manipulates resource identifiers to retrieve or modify another user's private data.
- **Business Logic Vulnerabilities including Race Conditions (A06:2025):** Flaws arising from incorrect assumptions in application workflow design, exploitable through precisely timed concurrent requests.
- **Broken Authentication Structures (A07:2025):** Structural weaknesses in token validation, including JWT algorithm confusion and session fixation vulnerabilities.

1.2 Why Is It Important?

As web applications grow in complexity, manual penetration testing cannot scale to match the pace of modern agile and continuous delivery development. The OWASP API Security Top 10 establishes that IDOR/BOLA alone accounts for the majority of API security incidents, and the Verizon Data Breach Investigations Report (2024) identifies authorization failure and business logic exploitation as the primary vectors for high-impact corporate data exfiltration.

Identifying these flaws remains heavily dependent on skilled human labor. Senior security engineers must manually operate intercepting proxies to map application state, construct multi-step hypothesis chains, and verify findings, a process that can consume dozens of hours per application. In modern CI/CD lifecycles this manual dependency constitutes a severe operational bottleneck, leaving complex logic vulnerabilities exposed in production environments due to the absence of scalable, automated reasoning tools. PentraAI directly addresses this capability gap by replacing rule-matching with LLM-driven contextual reasoning across the full penetration testing pipeline.

2. Overview of Existing Work

2.1 Deterministic Vulnerability Scanners

The dominant paradigm in automated web security testing is the deterministic scanner, exemplified by OWASP ZAP and Nuclei. These tools operate by fuzzing input parameters against known Common Vulnerability and Exposure (CVE) signatures and analyzing server responses for error patterns. Their primary strengths are speed, reproducibility, and low false-positive rates for structural defects such as missing security headers, reflected cross-site scripting, and basic SQL injection.

Their fundamental architectural weakness is the complete absence of semantic context. A deterministic scanner cannot model the relationship between two authenticated user sessions, nor can it reason about whether a server response constitutes an access-control violation or a legitimate application behavior. This limitation is not incidental; it is a structural consequence of the signature-matching paradigm itself (OWASP Top 10, 2025).

2.2 Reinforcement Learning Architectures

Early academic research explored Reinforcement Learning (RL) agents as an alternative to deterministic scanning, training models to navigate network topologies and maximize reward signals during exploitation tasks. While theoretically promising, RL-based approaches encounter a critical structural obstacle when applied to modern web applications: the sparse reward problem.

In a finite, well-defined environment an RL agent can learn effective exploitation paths through repeated trial and error. In the effectively infinite, unstructured state space of a production web application, where a correct multi-step business logic exploit may require hundreds of precisely sequenced, contextually aware HTTP transactions, the agent will almost never encounter a positive reward signal through random exploration. This renders standard RL architectures computationally intractable for the vulnerability classes PentraAI targets.

2.3 Conversational AI Security Assistants

The most recent paradigm shift involves integrating large language models (LLMs) into security workflows. Tools such as PentestGPT (Deng et al., 2023) and HackerGPT provide flexible, advanced reasoning capabilities through LLM prompt engineering, enabling them to analyses application descriptions and suggest contextually appropriate attack hypotheses.

However, as Deng et al. (2023) explicitly document, these systems operate as interactive advisory platforms rather than autonomous execution engines. The human operator must manually input application state, relay server responses back to the model, and physically execute recommended payloads. This human-in-the-loop architecture directly recreates the manual bottleneck it was intended to eliminate. Furthermore, Happe and Cito (2023) demonstrate that probabilistic LLMs operating without empirical grounding are prone to analytical hallucinations, asserting the presence of vulnerabilities based on statistical inference rather than verifiable evidence, producing an unacceptably high false-positive rate in uncontrolled advisory contexts.

3. Comparison of Approaches

3.1 Strengths and Weaknesses

Each existing paradigm addresses a subset of the problem while leaving critical gaps:

- **Deterministic Scanners (ZAP, Nuclei):** Highly efficient and fast with low false-positive rates for structural flaws such as missing headers and basic injection vulnerabilities. However, they possess a complete lack of semantic context, making them incapable of identifying when data exposure occurs across distinct user access controls or when application logic is abused.
- **Reinforcement Learning Architectures:** Well, optimized for path selection within narrow, predictable environments. However, they break down in the unstructured state spaces of modern web applications due to the sparse reward problem, where a model randomly guessing inputs will rarely execute a complex multi-step business logic sequence correctly.
- **Conversational AI Assistants (PentestGPT):** Provide flexible, advanced reasoning capabilities derived from LLM prompt engineering. However, because they lack an integrated programmatic execution engine, they rely on a manual human-in-the-loop workflow, which recreates the operational bottleneck they were designed to solve.

3.2 Signature-Based Scanning vs. Reasoning-Based Security Testing

The fundamental distinction between existing tools and PentraAI lies in their underlying epistemological model. Signature-based scanners operate deductively: they match observed behavior against a pre-existing library of known-bad patterns. This model is efficient for known

vulnerability classes but constitutionally incapable of identifying zero-day logic flaws or novel access-control violations, because no signature exists to match against.

Reasoning-based security testing, by contrast, operates inductively. PentraAI constructs a semantic model of the application under test, its parameters, resource identifiers, authentication structures, and workflow dependencies, and generates novel, contextually grounded hypotheses about potential failure modes. The fundamental distinction is between detection and reasoning: existing tools detect symptoms when a response matches a pattern, while PentraAI reasons about intent, reads the application, builds a model of what it is supposed to do, hypothesizes what could be abused, executes targeted tests, and interprets whether the response demonstrates actual exploitation. This mirrors how a skilled human penetration tester approaches a target rather than how a scanner does.

3.3 RL-Driven vs. LLM-Driven Agents

Reinforcement learning and LLM-driven agents represent two distinct approaches to introducing machine intelligence into security testing. RL agents learn optimal action sequences through environmental feedback, making them well suited to narrow, well-defined problem spaces with dense reward signals. LLM-driven agents leverage pre-trained semantic knowledge to reason about novel, unstructured environments without requiring environment-specific training.

For web application penetration testing, where the target application is unique, its logic is undocumented, and the vulnerability space is open-ended, the LLM-driven approach is demonstrably superior. The semantic knowledge embedded in a model such as Llama 3.3 encompasses thousands of known vulnerability patterns, HTTP protocol behaviors, and application logic archetypes, providing the contextual foundation required to generate meaningful hypotheses on first contact with a new target without any prior environment-specific training.

3.4 Human Pen testers vs. Autonomous AI Agents

The benchmark against which PentraAI must ultimately be assessed is the skilled human penetration tester. Expert security engineers bring irreplaceable capabilities: creative lateral thinking, the ability to model complex multi-stakeholder business logic, and accumulated domain intuition. PentraAI does not attempt to replicate this full capability. Its objective is to automate the systematic, hypothesis-driven component of expert methodology within the bounded scope of IDOR/BOLA, race condition, and broken authentication testing, positioning it not as a replacement for human expertise but as a scalable force multiplier that extends expert-level coverage to the CI/CD pipeline.

4. Identified Gap

4.1 What Is Missing?

There is no fully automated pipeline that can crawl an application, build a contextual model of how it works, hypothesize a complex attack, and then execute and verify it without a human clicking send. The three dominant paradigms each address part of the problem:

- Deterministic scanners provide execution without reasoning.
- RL agents provide learning without the semantic context required for modern web applications.
- Conversational AI assistants provide reasoning without execution.

No existing system unifies all three capabilities, contextual semantic reasoning, autonomous programmatic execution, and empirically grounded verification, into a single stateful closed-loop pipeline. This gap represents both the practical problem PentraAI solves and its primary academic contribution.

4.2 Where Is the Opportunity for Improvement?

Large language models provide the reasoning layer that was previously missing. By coupling LLM semantic intelligence with an agentic framework such as Lang Graph, it becomes possible to move beyond simple pattern matching and begin automating actual security thinking: reading an application, forming hypotheses about its failure modes, executing targeted tests, and interpreting empirical results.

The maturation of stateful multi-agent orchestration frameworks, specifically Lang Graph (Humeau et al., 2024), provides the architectural foundation for addressing this gap. Lang Graph enables the construction of persistent directed-graph execution environments in which specialized reasoning nodes share a common state vector, self-correct upon failure, and loop dynamically when intermediate results are inconclusive. This capability is precisely what is required to implement the multi-step, evidence-dependent logic that IDOR and authentication testing demands.

5. Positioning PentraAI

5.1 How Does the Project Address the Gap?

PentraAI implements a fully autonomous, five-phase agentic pipeline structured as a stateful directed graph. Each phase corresponds to a specialized Lang Graph node that processes a shared state vector, ensuring that contextual intelligence accumulated during reconnaissance is preserved and leveraged throughout the entire testing lifecycle:

- **Reconnaissance Node:** Five parallel discovery techniques map the full attack surface of the target application.

    - Common path probing to identify exposed endpoints and administrative interfaces.
    - Wayback Machine historical URL lookup to surface deprecated or forgotten routes.
    - Wordlist fuzzing across 200 or more API paths using curated security wordlists.
    - Subdomain enumeration via DNS resolution to identify scope-adjacent services.
    - Deep JavaScript analysis to extract embedded API calls from bundled frontend code.
- **Hypothesis Generation Node:** The LLM performs semantic analysis of the structured reconnaissance telemetry, generating prioritized, context-grounded vulnerability hypotheses rather than exhaustive payload lists.
- **Test Execution Node:** Hypotheses are translated into precise HTTP action sequences executed via asynchronous Python HTTP clients, including parameter mutations, cross-session token swaps, and synchronized concurrent request bursts.
- **Response Analysis Node:** A dedicated Dual-Verification Evidence Loop intercepts raw server responses. The agent cannot declare a finding valid without extracting concrete empirical proof from HTTP transaction data.
- **Remediation Reporting Node:** Confirmed findings are automatically mapped to CVSS v4.0 metrics and enriched with actionable developer remediation instructions.

5.2 What Approach Will You Take?

PentraAI is built on a Python and Fast API backend with a Lang Graph agent managing the five-phase pipeline. The LLM layer uses Llama 3.3-70b-versatile via the Grog API, with the frontend built on React and connected to the backend via Server-Sent Events for real-time streaming of scanning states, active security hypotheses, and confirmed vulnerabilities as they are detected.

The system specifically targets three OWASP Top 10:2025 categories where the automation and reasoning gap is largest: IDOR/BOLA (A01), Broken Authentication including JWT algorithm confusion and OAuth misconfigurations (A07), and Race Conditions (A06). The LLM is used only at decision points, hypothesis generation, response analysis, and remediation reporting, rather than for all tasks. This keeps inference costs near zero while maximizing the reasoning benefit where it matters most.

This architecture advances beyond PentestGPT along three distinct dimensions: it eliminates the human-in-the-loop execution bottleneck; it maintains a persistent structured state vector that prevents context degradation across multi-step logical tests; and it enforces empirical grounding to control the hallucination-driven false-positive rates documented by Happe and Cito (2023).

5.3 What Makes PentraAI State of the Art?

PentraAI introduces three quantifiable innovations over the current state of the art. First, it achieves full closure of the analysis-execution gap by removing all human intervention from the reconnaissance-through-verification loop. Second, its evidence-gated verification architecture provides a structural mechanism for false-positive control in LLM-driven security testing, a problem currently addressed only through post-hoc human review. Third, its A06:2025 Race Condition module implements single-packet synchronization techniques (PortSwigger Research, 2023) within an LLM-directed targeting framework, representing the first integration of precision temporal attack execution with AI-driven endpoint selection.

6. Research to Application Mapping

6.1 LLMs Hallucinate in Security Contexts Without Grounding

Happe and Cito (2023) provide the most rigorous analysis of LLM behavior under adversarial security testing conditions, demonstrating that without explicit grounding constraints, LLMs generate false-positive vulnerability assertions at rates that render advisory outputs operationally unreliable. Their findings identify prompt-level context window limitations and statistical inference patterns as the primary failure mechanisms.

Application: PentraAI implements a strict Response Analysis Phase enforcing a Dual-Verification Evidence Loop. The reasoning agent cannot declare a vulnerability valid unless it extracts concrete empirical proof from raw HTTP transaction data, such as specific leaking data records, clear server response variations, or explicit cryptographic anomalies parsed directly from raw HTTP transaction strings. This design directly mitigates the risk of false positives documented in the literature.

6.2 Effective Race Condition Testing Requires Precise Timing Synchronization

PortSwigger Research (2023) establishes that reliable race condition discovery requires sub-millisecond concurrent request synchronization using single-packet or last-byte alignment techniques to compress request execution within tight network timing windows.

Application: The A06:2025 module uses the LLM to analyses endpoint paths and identify transaction workflows susceptible to concurrency exploitation, such as wallet withdrawal endpoints, coupon redemption flows, or inventory reservation handlers. Identified targets are programmatically passed to an optimized asynchronous Python routine using HTTP that coordinates synchronized concurrent network request bursts, directly implementing the single-packet timing technique specified in the PortSwigger research.

6.3 Multi-Step Reasoning Requires Persistent State Management

Academic literature on stateful agent architectures confirms that open-ended conversational frameworks degrade over extended multi-step reasoning tasks as context windows fill and earlier state is lost. Effective autonomous agents require a persistent, structured state vector that is explicitly maintained and passed through each reasoning phase (Humeau et al., 2024).

Application: PentraAI uses Lang Graph to allow the agent to loop back and re-test if a hypothesis is not initially proven. Instead of relying on an open-ended conversational thread that degrades over time, the system uses a persistent structured state vector that flows through specialized sub-graphs to maintain absolute context during multi-step logical testing. This enables the agent to self-correct, update its hypotheses, and retry dynamically when automated tests fail.

6.4 IDOR/BOLA Detection Requires Multi-Session Authorization Modelling

Academic literature confirms that reliable identification of access control anomalies such as IDOR/BOLA requires modelling multi-user authorization structures and tracing resource identifier parameters across independent authenticated sessions (OWASP API Security, 2023). Standard scanners cannot perform this analysis because they operate with a single session context.

Application: The A01:2025 module provisions and maintains two distinct, fully authenticated user sessions within the Lang Graph shared state. During the test execution phase, the agent systematically manipulates object keys identified during reconnaissance such as tenant, invoice Num, and user_uuid, issuing cross-account HTTP requests and analyzing responses for data leakage indicative of authorization failure. This directly operationalizes the multi-session modelling approach recommended in the OWASP API Security literature.

6.5 JWT Vulnerability Classes Require Targeted Structural Analysis

Celiere et al. (2022) provide a comprehensive taxonomy of JWT attack vectors, identifying algorithm confusion (alg: none substitution), weak secret exploitation, and privilege escalation via claim manipulation as the three primary structural vulnerabilities in token-based authentication systems. Each requires a distinct test methodology that cannot be captured by generic input fuzzing.

Application: The A07:2025 Broken Authentication module uses the LLM to parse token structures observed during reconnaissance, identify the signing algorithm in use, and generate targeted mutation payloads corresponding to each attack class identified by Celiere et al. (2022). This academic grounding ensures the authentication testing module covers the full taxonomy of known JWT vulnerabilities rather than relying on ad hoc payload lists.

7. AI Safety, Ethics, and Validation

7.1 Responsible Deployment of Autonomous Offensive Security Systems

The deployment of autonomous offensive security tools introduces distinct ethical and safety considerations that extend beyond standard software engineering practice. Because PentraAI executes real HTTP requests against live application endpoints, the risk of unintended disruption, misuse, or data exposure requires explicit architectural mitigation rather than policy-level guidance alone.

PentraAI incorporates three structural safety controls. First, dual-use isolation decouples the output layer from arbitrary exploit payload execution; the system is architecturally constrained to security posture diagnostics, risk reporting, and remediation guidance, and cannot be trivially reconfigured as a weaponized attack tool. Second, stability enforcement constrains the execution client with configurable rate limits, absolute spider depth boundaries, and a complete block on destructive HTTP mutation verbs unless explicitly authorized in the session configuration. Third, scope confinement ensures the agent operates exclusively within the target URL space defined at session initialization, preventing lateral scope creep into adjacent systems.

7.2 Validation Methodology and Quantitative Benchmarks

To establish rigorous academic validity, PentraAI is evaluated exclusively against standardized, purpose-built vulnerable reference environments: the Damn Vulnerable Web Application (DVWA) and the OWASP Juice Shop. These environments provide a fixed, ground-truth vulnerability set that enables objective, reproducible measurement.

The primary performance metrics are:

- **Detection Rate (DR):** The ratio of true positive findings to the total number of independently verified vulnerabilities presents in the target environment, expressed as a percentage. Target threshold: DR of 80 percent or above across all three vulnerability categories (A01, A06, A07).
- **False Positive Rate (FPR):** The ratio of incorrectly asserted findings to total independent test iterations, expressed as a percentage. Target threshold: FPR of 5 percent or below across all test suites.

These thresholds are benchmarked directly against the published performance characteristics of OWASP ZAP and Nuclei on equivalent logic-flaw test cases, providing a grounded comparative baseline. Statistical significance is established through a minimum of 30 independent test runs per vulnerability category. Additionally, parameter capture completeness, the percentage of injectable parameters successfully identified during reconnaissance, is tracked as a prerequisite metric for downstream detection accuracy. The empirical evaluation also compares True Positive ratios directly against standard tools under identical logic test scenarios and measures the agent's ability to isolate session context mismatches without generating invalid HTTP requests.

8. Key References

OWASP Top 10. (2025). *A Standardized Awareness Document for Web Application Security.* Primary taxonomy framework defining target vulnerability categories and scope boundaries for PentraAI's testing modules.

OWASP API Security Top 10. (2023). *API Security Awareness Document.* Detailed attack patterns for IDOR/BOLA (API1) and Broken Authentication (API2); direct academic grounding for the multi-session authorization modelling approach used in the A01:2025 module.

Deng, G., et al. (2023). *PentestGPT: Evaluating Large Language Models for Automated Penetration Testing.* Foundational study documenting the operational limits, context degradation behaviors, and human-in-the-loop bottlenecks of conversational AI security assistants. Primary comparative benchmark against which PentraAI's autonomous execution contribution is assessed.

Happe, A., and Cito, J. (2023). *Getting pwn'd by AI: Penetration Testing with Large Language Models.* Core empirical research characterizing hallucination distribution profiles, false-positive generation mechanisms, and prompt engineering failure modes in LLMs under adversarial conditions. Direct motivation for PentraAI's Dual-Verification Evidence Loop.

PortSwigger Research. (2023). *Race Conditions: New Classes and the Single-Packet Attack. Black Hat USA 2023.* Technical reference defining packet alignment methods, concurrency timing window verification, and last-byte synchronization techniques. Foundation for PentraAI's race condition timing implementation.

Humeau, M., et al. (2024). *Lang Graph: Building Stateful Multi-Actor Applications with LLMs.* System architecture reference for directed-graph state management, multi-node orchestration, and resilient agent feedback loops. Foundational framework underpinning PentraAI's pipeline architecture.

Celiere, J., et al. (2022). *JWT Security Taxonomy and Common Attack Vectors.* Comprehensive reference for the Broken Authentication module design, covering algorithm confusion (alg: none), weak secret exploitation, and privilege escalation via claim manipulation.

Verizon. (2024). *Data Breach Investigations Report (DBIR).* Enterprise risk dataset establishing the empirical prevalence and business impact of authorization failure and business logic exploitation as primary breach vectors.

Grog Inc. (2025). *Llama 3.3-70b-versatile API Documentation.* Technical reference for the LLM inference layer used across all three reasoning phases in PentraAI.

9. Conclusion

9.1 What the Literature Confirms

The reviewed literature collectively establishes four findings that directly shape PentraAI's architecture. First, deterministic pattern-matching automation is a solved challenge for shallow structural vulnerabilities but represents an architectural dead-end for logic-driven, context-sensitive flaws. Second, early AI security systems fail in practice because they are designed as open-ended conversational interfaces rather than stateful, goal-oriented execution pipelines, recreating the same human-in-the-loop bottleneck they purport to eliminate. Third, probabilistic LLMs without explicit empirical grounding mechanisms generate operationally unacceptable false-positive rates, making autonomous deployment infeasible without a structural verification constraint. Fourth, the technical foundations for reliable race condition exploitation are well-established and amenable to programmatic automation, representing an untapped opportunity for LLM-directed targeting combined with precision asynchronous execution.

While AI is increasingly being used as a security assistant, the jump to a fully autonomous agent remains new and technically difficult, especially regarding logic flaws. PentraAI represents a direct attempt to close this gap through principled architectural design rather than incremental tool improvement.

9.2 How the Literature Shaped Our Direction

The research confirmed the decision to focus exclusively on the three OWASP categories with the largest automation gap: IDOR (A01), Broken Authentication (A07), and Race Conditions (A06), rather than building another scanner for SQL injection or XSS where ZAP and Nuclei already perform well. It also validated the architectural decision to use LLM reasoning only at decision points, hypothesis generation, analysis, and reporting, rather than for all tasks, keeping inference costs near zero while maximizing the reasoning benefit.

The PortSwigger race condition research directly shaped the implementation of the timing attack module. The hallucination findings of Happe and Cito (2023) made the evidence-gated verification architecture a non-negotiable design requirement. The Lang Graph framework was selected over linear alternatives specifically because its persistent shared state vector enables the multi-session authorization modelling required for IDOR detection.

Most importantly, the literature confirmed that the five-phase autonomous pipeline PentraAI implements, Reconnaissance, Hypothesis Generation, Test Execution, Response Analysis, and Remediation Reporting, represents a genuinely novel contribution rather than an incremental improvement on existing tools. PentraAI's contribution is the first end-to-end autonomous pipeline to unify LLM-driven semantic hypothesis generation, programmatic HTTP execution, and empirically grounded verification into a closed stateful loop, directly operationalizing the methodology of expert human penetration testers within a scalable, CI/CD-compatible system.

10. Readability Statistics